

Exercício 5: Construtores, Encapsulamento e Integridade

Nesta jornada, vamos aprender como garantir que nossos objetos "nasçam" e "vivam" de forma correta e segura. Vamos transformar o código em um sistema robusto contra erros!

1. O Nascimento do Objeto: Métodos Construtores

Imagine a linha de montagem de um **Smartphone**. Antes do aparelho sair da fábrica, ele precisa ser configurado com um número de série e um sistema operacional. Ele não pode sair "vazio".

Na programação, o **Construtor** é um método especial que é executado automaticamente no momento em que você cria o objeto (usando a palavra `new`).

Para que serve?

- Garantir que o objeto comece com dados válidos.
- Obrigar quem usa a classe a passar as informações essenciais.

Regra Visual: O construtor sempre tem o **mesmo nome da Classe** e não possui tipo de retorno (nem mesmo `void`).

2. O Conceito: Encapsulamento (A "Casca" Protetora)

Imagine um **Copo Térmico**. Você sabe que ele mantém o café quente, mas você não precisa ver o vácuo entre as paredes de aço ou entender a física térmica para usá-lo. Você interage apenas com a **tampa**.

O Encapsulamento faz exatamente isso:

1. **Esconde o que é sensível** (usa o modificador `private`).
2. **Cria uma interface de uso** (métodos `public` como Getters, Setters e Construtores).

Por que usar validação?

O maior benefício é **controlar como o dado muda**. Se um atributo é `public`, qualquer um pode colocar um valor absurdo nele. Com o encapsulamento, criamos um "segurança" (o método) que verifica o valor antes de permitir a alteração.

3. Mão na Massa: Sistema de Cadastro de Produtos

Você está desenvolvendo um sistema para uma loja. Vamos garantir que o produto já nasça com um nome e que o preço seja sempre protegido.

Sua Missão:

3. Complete o **Construtor** para receber o nome e o preço inicial.
4. Mude a visibilidade do atributo `preco` para `private`.
5. No método `setPreco`, adicione a lógica de validação para impedir preços negativos.

Complete o código Java abaixo:

```
public class Produto {
    private String nome;
    // 1. Torne o atributo preco PRIVADO
    public double preco;

    // 2. CONSTRUTOR: Complete para receber nome e preco
    public Produto(String nome, double precoInicial) {
        this.nome = nome;
        // DICA: Use o método setPreco aqui também para validar o
início!
        this.setPreco(precoInicial);
    }

    // 3. Método SETTER com VALIDAÇÃO
    public void setPreco(double novoPreco) {
        if (novoPreco > 0) {
            this.preco = novoPreco;
            System.out.println("Preço de '" + nome + "'
atualizado!");
        } else {
            System.out.println("Erro: Preço inválido para o
produto '" + nome + "'!");
        }
    }

    // 4. Método GETTER
    public double getPreco() {
        return this.preco;
    }
}
```

4. Reflexão de Aprendizado I

Pergunta: Se você usar o método setPreco dentro do seu **Construtor**, o que acontece se alguém tentar criar um produto novo com preço -50.00? O sistema estaria protegido logo no "nascimento" do objeto?

5. Desafio Final: Autonomia Total

Agora, construa um sistema do zero aplicando tudo o que aprendeu: Construtor, Atributos Privados e Validação.

Cenário: Você foi designado para criar o software de um **Termostato Inteligente** para uma sala de servidores.

- A temperatura da sala **nunca** pode ser menor que 15°C e nem maior que 30°C.

Sua Missão:

Crie a classe Termostato seguindo estas regras:

1. Atributo temperaturaAtual deve ser **privado**.
2. Crie um **Construtor** que obrigue a definição de uma temperatura inicial.

3. Crie um método `ajustarTemperatura(double novaTemp)` com a lógica de proteção (15 a 30).
4. Crie o método **Getter** para leitura.

Escreva seu código abaixo:

```
// Implemente sua classe Termostato completa assim...
public class Termostato {
    // Atributos...

    // Construtor...

    // Getters e Setters com validação...
}
```

```
// Classe para testar
public class TesteTermostato {
    public static void main(String[] args) {
        // Teste criar com temperatura válida e inválida
        Termostato meuTermo = new Termostato(20.0);

        meuTermo.ajustarTemperatura(10.0); // Deve ser bloqueado!

        System.out.println("Temperatura atual: " +
meuTermo.getTemperaturaAtual());
    }
}
```

6. Reflexão Final de Materialização

Pergunta Final: Imagine que esse sistema será usado em 10 computadores diferentes. Se a regra de temperatura mudasse para 18°C a 28°C, qual a vantagem de ter essa regra escondida dentro da classe `Termostato` (encapsulada) em vez de espalhada pelo código principal de todos os computadores?

Parabéns! Você agora domina o ciclo de vida e a segurança dos dados em um objeto!